

CPP NOTES – DAY 24

Structs cont...

Calloc

- Allocates memory for an array of elements, initializing all bytes to zero.
- "calloc" = "contiguous allocation"
- *Eg: `int *arr = (int *)calloc(5, sizeof(int));`*

Realloc

realloc is used to **resize** a previously allocated block of memory (via malloc, calloc, or even realloc itself) without losing the existing data.

Eg: `int *arr = (int *)malloc(5 * sizeof(int));`

`arr = (int *)realloc(arr, 10 * sizeof(int)); // Resize to hold 10 ints`

New

new is a C++ operator used to allocate memory dynamically on the heap, and initialize an object or a primitive type.

PADDING

- **Padding** refers to the process of adding extra unused memory spaces within a **data structure** (like a **struct** or **class**) to ensure proper memory alignment.
- Memory alignment is crucial for efficient access to data in memory, especially on architectures where data needs to be aligned to certain byte boundaries (e.g., 4-byte or 8-byte boundaries).

UNION

A union is a special data structure in C++ that allows you to store different data types in the same memory location. However, unlike a struct, where each member has its own memory space, a union shares memory for all its members.

This means that only one of the union members can hold a value at a time. The memory required for the union is determined by the size of its largest member.

Feature	Description
Memory Usage	Shares memory space among all its members
Size	Size is equal to the size of its largest member
Initialization	Only one member can hold a value at any given time
Use Case	Memory optimization, type-punning, handling variant types

FYI

- *The copying in struct can be done using = operator.
Eg: s1.length=s2.length*
- *If I declare a struct inside another struct, the inner struct will be tight bounded to outer struct*
- *When realloc used to reallocate already defined block using malloc:*
- *If the existing memory block has enough space to grow, realloc may resize it **in place**, and the base address stays the same.*
- *If not, it will:*
 1. *Allocate a new block of the requested size.*
 2. *Copy the contents of the old block to the new one.*
 3. *Free the old block.*
 4. *Return the new base address.*
- *Pointer to pointer allocation – whenever there is no rows and col value which have the potential to grow dynamically.eg: Hashing (but slow in performance)*

- Flat 1d arr – fixed no of cols and rows. Is low in flexibility(cant grow dynamically) but high in performance.
- Stack frames are not generated for **inline functions**.
- `int &a = 101` cause compilation error because: A reference must bind to a variable (an lvalue), not a literal (a rvalue)
- que: What is the diff between rvalue and rvalue reference
- `cout<<"Hello"<<endl;` this gets printed because of **rvalue reference**. Therefore all of the literals/rvalues have a rvalue reference along.
- Constants can only be printed not user defined